

位运算的由来

在计算机里面，任何数据最终都是用数字来表示的（不管是我们平时用的软件，看的图片，视频，还是文字）。
并且计算机运算单元只认识高低电位，转化成我们认识的逻辑，也就是 0 1。
这就是导致计算机里面任何数据最终都是用二进制（0 1）来保存的数字。只是我们平时看到的图片、文字、软件都只从二进行数字转化而来的。

位运算符

符号	描述	运算规则
&	与	两位都为1，那么结果为1
	或	有一位为1，那么结果为1
~	非	$\sim 0 = 1, \sim 1 = 0$
^	异或	两位不相同，结果为1
<<	左移	各二进位全部左移若干位，高位丢弃，低位补0
>>	右移	各二进位全部右移若干位，对无符号数，高位补0，有符号数，各编译器处理方法不一样，有的补符号位（算术右移），有的补0（逻辑右移）

常用位操作

1. 判断奇偶

- $(x \& 1) == 1$ ---等价---> $(x \% 2 == 1)$
- $(x \& 1) == 0$ ---等价---> $(x \% 2 == 0)$

- 2. $x / 2$ ---等价---> $x >> 1$
- 3. $x \&= (x - 1)$ -----> 把x最低位的二进制1给去掉
- 4. $x \& -x$ -----> 得到最低位的1

5. $x \ \& \ \sim x \text{ ----} \rightarrow 0$

指定位置的位运算

1. 将x最右边的n位清零： $x \& (\sim 0 \ll n)$
2. 获取x的第n位值： $(x \gg n) \& 1$
3. 获取x的第n位的幂值： $x \& (1 \ll n)$
4. 仅将第n位置为1： $x | (1 \ll n)$
5. 仅将第n位置为0： $x \& (\sim(1 \ll n))$
6. 将x最高位至第n位（含）清零： $x \& ((1 \ll n) - 1)$
7. 将第n位至第0位（含）清零： $x \& (\sim((1 \ll (n + 1)) - 1))$

异或结合律

$$\begin{aligned}x \wedge 0 &= x, & x \wedge x &= 0 \\x \wedge (\sim 0) &= \sim x, & x \wedge (\sim x) &= \sim 0 \\a \wedge b &= c, & a \wedge c &= b, & b \wedge c &= a\end{aligned}$$

(有没有点乘法结合律的意思)

字母表示： $(a \wedge b) \wedge c = a \wedge (b \wedge c)$

图形表示： $(\star \wedge \diamond) \wedge \triangle = \star \wedge (\diamond \wedge \triangle)$

大小字母位运算技巧

(2021-05-09 更新)

- 大写变小写、小写变大写：字符 ^ = 32 （大写 ^ = 32 相当于 +32，小写 ^ = 32 相当于 -32）
- 大写变小写、小写变小写：字符 | = 32 （大写 | = 32 就相当于+32，小写 | = 32 不变）
- 大写变大写、小写变大写：字符 & = -33 （大写 ^ = -33 不变，小写 ^ = -33 相当于 -32）

具体为什么是 32 -33 这2个数字，详见本人写的第709题，题解：明明只有26个字母,为什么大小字母之间是差32而不是26？

正式解题

2 的幂次方，也就是： 2^0 、 2^1 、 2^2 、 $\dots$

n 是 int 类型，高最位是符号，也就除最高位，于下的31位，里面只有1个1

[illegible]

解法1(相当于乘法)

通过左位移操作，列举出 int 型内的全部2的幂次方

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        if (!n) return false;
        for (int i = 1, sub = 1; i < 32; ++i, sub <<= 1)
            if (sub == n) return true;
        return false;
    }
};
```

解法2(相当于除法)

通过右位操作，把n做2的幂次方的逆逻辑

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        if (!n) return n;
        while (n % 2 == 0) n >>= 1;
        return n == 1;
    }
};
```

解法1、2是通过数字的位移操作

解法3

利用上面掉到的常用位操作：

1. $n \& -n$ 得到最低位的 1 的位置

2. 再和 $n \wedge$

如果 n 只有一个1，那么 $n \wedge (n \& -n)$ 就只可能是0，要么非0

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        return n > 0 && (n ^ (n & -n)) == 0;
    }
};
```

解法4

直接利用上面掉到的常用方法 $n \& (n - 1)$ 去除最低位的1。

如果 n 的二进制里面只有一个1，那么去掉最低位的1，就只可能是0，要么非0

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        return (n > 0 && (n & (n - 1)) == 0);
    }
};
```

小结1：

上面4种解法的本质，其实就是求出32个bit位里面有几个1，有点类似于JAVA里面的

```
return Integer.bitCount(n) == 1;
```

C++里面有也类似的，个人感觉写出来没有JAVA的好看

解法5，特化解法

传入的参数 n 为 `int` 类型

也就是 $-2^{31} \leq n < 2^{31}$

✓如果 n 为 2 的幂次方的话一定是：

$[1, 2^{30}]$ 双闭区间

把 n 作为除数， 2^{30} 作为被除数，那么，如果 n 是 2 的幂次方的话，余数肯定为0

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        return n > 0 && (1 << 30) % n == 0;
    }
};
```

小结2

其实很多网上有很多有关代码的奇技淫巧都有位操作的影子，太窄的应用方式个人觉得真的没有必要花费时间和精力去记。把上面那些平时使用比较高的记下来就可以了。我猜力扣出有关二进制的目的也是这样的。